

项目上传与协作开发指南

- **User:** topologicalstructure
- **Start:** 2026-05-08 16:37:49
- **End / Last Change:** 2026-05-09 13:31:51

Request:

- **Mode:** Agent

这是一个行动导向型辅导系统，请参考README文件的设计补全代码

Response:

- **Model:** GPT-5.3-Codex

收到，我已经按 README 的设计把 MVP 主链路补全，并先跑了本地冒烟测试验证可用。

- [x] 对齐 Trace -> Gap -> Nudge 数据契约，补齐后端核心模块
- [x] 实现 Flask 三个主接口： /upload_trace 、 /detect_gap 、 /generate_nudge
- [x] 接入 Gradio 前端，支持输入学习轨迹并展示 gaps + nudge
- [x] 增加最小 smoke test，验证接口与端到端流程
- [x] 补充缺失 prompt 文件内容，保证提示词链路可运行

我具体完成了这些代码补全：

- app.py
 - 新增 create_app()，组装 EmbeddingModel、ChromaStore、TraceIndexer、GapDetector、LLMClient、NudgeEngine
 - 新增路由： GET /health、POST /upload_trace、POST /detect_gap、POST /generate_nudge
 - 新增统一异常 JSON 返回
- backend/embed_model.py
 - 替换原实验脚本为 EmbeddingModel 类
 - 实现轻量、可离线运行的确定性 hash embedding（避免重依赖）
- backend/chroma_store.py
 - 实现 ChromaStore 类
 - 支持 chromadb 持久化存储

- 如果 chromadb 不可用, 自动降级为内存存储 (保证 MVP 可跑)
- backend/trace_indexer.py
 - 实现 `TraceIndexer.process(trace)`
 - 解析 `notes/exercise_logs/error_history`
 - 统计 `attempts/mistake_count/error_rate/mastery/last_seen/revisit_gap_days`
 - 写入向量存储并返回结构化 `concepts + trace_summary`
- backend/gap_detector.py
 - 实现 `GapDetector.detect(indexed_trace)`
 - 按 README 的启发式权重计算 gap score:
 - `error_rate`、`forgetting_score`、`avoidance_score`、`hesitation_score`
 - 返回排序后的 `weak_concepts`
- backend/llm_client.py
 - 实现 `LLMClient` (可选 OpenAI)
 - 有 `OPENAI_API_KEY` 时走 chat completion
 - 无 key 时自动 fallback 生成可行动 nudge (确保可运行)
- backend/nudge_engine.py
 - 实现 `NudgeEngine.generate(...)`
 - 读取 `prompts/nudge_system.txt`
 - 基于最弱 concept 生成目标 nudge, 输出 JSON-safe
- frontend/gradio_ui.py
 - 构建可运行 Gradio 页面
 - 支持输入 `notes / exercise_logs(JSON) / error_history(JSON)`
 - 调用后端 `/generate_nudge` 并展示 `weak_concepts` 与 `nudge`
- prompts/gap_analysis.txt
 - 补充基础 gap 分析提示词模板 (之前为空)
- backend/__init__.py
 - 新增包初始化文件, 完善模块结构
- tests/test_pipeline_smoke.py
 - 新增 2 个冒烟测试:
 - `test_generate_nudge_pipeline`
 - `test_upload_trace`

已验证结果（我本地执行）：

- `python -m unittest discover -s tests -v` 通过 (2/2)

你现在可以直接这样跑：

```
cd /d D:\source\PycharmProjects\action_tutor
python app.py
```

```
cd /d D:\source\PycharmProjects\action_tutor
python frontend\gradio_ui.py
```

如果你愿意，我下一步可以做这 2 个增强（选一个数字就行）：

1. 把 `GapDetector` 的分数和原因解释成更可视化的诊断字段（便于课程展示）
2. 增加 `data/traces` 样例 JSON + 一键演示脚本，让你 demo 时直接加载数据

Request:

将嵌入编码改为使用本地 `sentence-transformers/all-MiniLM-L6-v2`，参考 `test_embed.py`；另外，大模型调用的 `OPENAI_BASE_URL`、`OPENAI_MODEL` 等改为从 `.env` 中读取

Response:

- **Model:** GPT-5.4 mini

已完成这次调整，核心已经切到本地 MiniLM 嵌入，并把 OpenAI 配置改成从 `.env` 读取。

已完成项

- [x] `backend/embed_model.py` 改为本地 `sentence-transformers/all-MiniLM-L6-v2` 路线
 - 使用 `AutoTokenizer` + `AutoModel`
 - 按 `test_embed.py` 的方式做 `mean pooling` + `normalize`
- [x] `backend/llm_client.py` 改为从环境变量读取：
 - `OPENAI_API_KEY`
 - `OPENAI_BASE_URL`
 - `OPENAI_MODEL`
- [x] `app.py` 中的 Chroma 集合名更新为与新 embedding 维度兼容，避免旧 128 维数据冲突
- [x] `tests/test_embed.py` 增加主入口保护，避免 `unittest discover` 时误执行示例脚本
- [x] 重新跑通测试，确认没有回归

现在的行为

- 嵌入向量来自本地 `sentence-transformers/all-MiniLM-L6-v2`
- 大模型调用会优先使用 `.env` 中的配置
- 没有配置或调用失败时，仍保留 `fallback nudge`，保证 MVP 可运行

验证结果

我已本地执行并通过：

```
cd /d D:\source\PycharmProjects\action_tutor
python -m unittest discover -s tests -v
```

如果你愿意，我下一步可以继续帮你做两件事里的任意一个：

1. 把 `.env` 配置项补到 `README.md`
2. 给 `embed_model.py` 再加一个批量 `embed_texts()` 接口，后面做 Chroma 批量入库会更方便

Request:

我现在要将该项目上传至GitHub以便协作开发，请帮助我重构一下README文件（英语），需要包含但不限于项目介绍、架构设计、功能设计、环境配置（包含`.env`）、项目运行说明、项目完成进展、下一步计划等。内容尽可能全面，以便他们可以快速配置环境并借助AI Agent进一步开发。

Response:

- **Model:** GPT-5 mini

I will update the repository README to a comprehensive English developer guide and onboarding document, then add instructions for setup, `.env`, running, testing, progress, and next steps.

I have replaced the project's README with an English README that includes:

- Project overview and architecture
- File/component map and responsibilities
- Environment setup and `.env` content
- How to run tests, the Flask API and Gradio UI
- Example API usage
- Current project progress (what's implemented)
- Troubleshooting notes (including Chroma embedding-dimension guidance)
- Contribution and next-step roadmap

If you want, I can next:

1. Add a Dockerfile + docker-compose for convenient local orchestration, or

2. Add GitHub Actions CI that runs the test suite and optionally builds the Docker image.

Which would you like me to implement next?

Request:

我现在要将该项目上传至GitHub以便协作开发，请帮助我重构一下README文件（英语），需要包含但不限于项目介绍、架构设计、功能设计、环境配置（包含.env）、项目运行说明、项目完成进展、下一步计划等。内容尽可能全面，以便他们可以快速配置环境并借助AI Agent进一步开发。

Response:

- **Model:** GPT-5.2-Codex

Got it. I'm replacing the README with a complete English onboarding guide tailored for GitHub collaboration.

Checklist

- [x] Rewrite README.md in English
- [x] Include overview, architecture, features, setup, .env , running, progress, and next steps
- [x] Keep it clear and actionable for new collaborators and AI agents

I updated README.md with a full English guide that covers:

- Project overview and goals
- Architecture and component responsibilities
- Feature breakdown (Trace → Gap → Nudge)
- Environment setup and .env configuration
- How to run tests, API, and UI
- Current progress and known constraints
- Next steps and AI-agent-friendly collaboration tips

If you want, I can also add:

1. A Dockerfile + docker-compose.yml for one-command setup
2. A GitHub Actions CI workflow that runs tests on PRs

Tell me which one to add next.

End of Chat